

几何盛宴，震撼美味！

2026.5.2 4:03 cc补题单剩下的感觉全是不可做题啊.....不兑！怎么还有十几道几何，到底是谁塞进去的
来场几何盛宴，以下是2024年至今XCPC的所有几何题

签到蛆

2024ICPC昆明H: [Horizon Scanning - Problem - QOJ.ac](#) 

铜牌蛆

2025EC-FINAL杭州A: [Outstanding Outlines - Problem - QOJ.ac](#)

银牌区

2024ICPC沈阳G: [Guess the Polygon - Problem - QOJ.ac](#)

2024CCPC哈尔滨B: [Concave Hull - Problem - QOJ.ac](#)

2025ICPC南京B: [What, More Kangaroos? - Problem - QOJ.ac](#)

2025ICPC沈阳G: [Collision Damage - Problem - QOJ.ac](#) 

金牌区

2024ICPC香港E: [Concave Hull - Problem - QOJ.ac](#)

2024ICPC南京M: [Ordainer of Inexorable Judgment - Problem - QOJ.ac](#)

2025ICPC成都I: [Inside Triangle - Problem - QOJ.ac](#)

2025ICPC南京M: [Many Convex Polygons - Problem - QOJ.ac](#)

2025ICPC武汉G: [Projection - Problem - QOJ.ac](#)

2025CCPC重庆C: [GPA Optimization Plan - Problem - QOJ.ac](#)

捧杯区+无人区

2024ICPC成都M: [Two Convex Holes - Problem - QOJ.ac](#)

2024ICPC杭州C: [Catch the Star - Problem - QOJ.ac](#)

2024ICPC上海A: [Ancient Maps, Hidden Danger - Problem - QOJ.ac](#)

2024ICPC上海M: [Machine Learning with Penguins - Problem - QOJ.ac](#)

2024ICPC沈阳K: [Fragile Pinball - Problem - QOJ.ac](#)

2024EC-FINAL西安D: [Keystone Correction - Problem - QOJ.ac](#)

2024CCPC济南G: [The Wheel of Fortune - Problem - QOJ.ac](#)

2025ICPC成都E: [Escaping from Trap - Problem - QOJ.ac](#)

2025ICPC南京L: [Regional Champion - Problem - QOJ.ac](#)

2025ICPC上海F: [Flower's land 4 - Problem - QOJ.ac](#)

2025CCPC济南B: [Fortress - Problem - QOJ.ac](#)

Horizon Scanning

来源：2024ICPC昆明H 难度：签到

题意：给定平面上的 n 个点，要求在原点放置一个雷达，使得雷达在旋转至任意角度时，都能照到至少 k 个点。最小化雷达照射的角度范围

数据范围： $1 \leq T \leq 10^4$, $1 \leq k \leq n \leq 2 \times 10^5$, $|x_i|, |y_i| \leq 10^9$, $\sum n \leq 2 \times 10^5$

题解：使用 `atan2` 等方法，将点转化为极角。此时问题转变为，求一个循环的极角序列上，相隔 k 个位置的极角差值的最大值

```
1 using db = long double;
2 const db pi = acos(-1);
3 const db eps = 1e-8;
4
5 int dcmp(db x, db y) {
6     if(fabs(x - y) < eps) return 0;
7     return x > y ? 1 : -1;
8 }
9
10 void solve() {
11     int n, k;
12     cin >> n >> k;
13     vector<db> a(n);
14     for(int i = 0; i < n; i++) {
15         int x, y;
16         cin >> x >> y;
17         a[i] = atan2(x, y);
18     }
19     sort(a.begin(), a.end());
20     for(int i = 0; i < n; i++)
21         a.push_back(a[i] + pi * 2.0);
22
23     db res = 0;
```

```

24     for(int i = 0; i + k < n * 2; i++)
25         if(dcmp(a[i + k] - a[i], res) == 1)
26             res = a[i + k] - a[i];
27     cout << fixed << setprecision(12) << res << '\n';
28 }

```

Collision Damage

来源：2025ICPC沈阳G 难度：银牌

题意：给定凸多边形 P 和 Q ，在 $area(P \cap (Q + t)) > 0$ 的前提下均匀选择 t ，求 $area(P \cap (Q + t))$ 的期望

数据范围： $1 \leq T \leq 300$ ， $3 \leq n, m \leq 1000$ ， $-10^4 \leq x, y \leq 10^4$ ， $\sum n, \sum m \leq 2 \times 1000$

题解：本质是求， P 和 Q 有交的前提下， Q 均匀随机出现在平面上， P 与 Q 交集面积的期望。交集总面积（即分子）为 P 和 Q 面积的乘积，分母为 P 和 Q 有交的单位时间（用时间这个概念可能有点抽象，反正就是那个意思），可以发现是 P 和 $-Q$ 的闵可夫斯基和（ Q 接触着 P 尽可能大的绕一圈），可以将 P 和 Q 的顶点两两做差后对所有点求凸包面积

时间复杂度：闵可夫斯基和： $O(n + m)$ ，凸包 $O(nm \log(nm))$

标签：凸包，闵可夫斯基和

```

1  using ll = long long;
2  using db = long double;
3
4  // =====
5  // 1. 点与向量模版 (Point & Vector)
6  // =====
7  struct Point {
8      ll x, y;
9      Point(ll _x = 0, ll _y = 0) : x(_x), y(_y) {}
10
11      Point operator+(const Point& p) const { return Point(x + p.x, y + p.y); }
12      Point operator-(const Point& p) const { return Point(x - p.x, y - p.y); }
13
14      bool operator<(const Point& p) const {
15          if (x != p.x) return x < p.x;
16          return y < p.y;
17      }
18      bool operator==(const Point& p) const {
19          return x == p.x && y == p.y;
20      }
21 };
22

```

```

23 // =====
24 // 2. 核心几何函数
25 // =====
26
27 // 向量外积 (叉乘)
28 ll cross(const Point& a, const Point& b) {
29     return a.x * b.y - a.y * b.x;
30 }
31
32 // 三点外积
33 ll cross(const Point& o, const Point& a, const Point& b) {
34     return cross(a - o, b - o);
35 }
36
37 // 向量内积 (点乘)
38 ll dot(const Point& a, const Point& b) {
39     return a.x * b.x + a.y * b.y;
40 }
41
42 // 三点内积
43 ll dot(const Point& o, const Point& a, const Point& b) {
44     return dot(a - o, b - o);
45 }
46
47 /**
48  * To-Left 测试
49  * 返回值:
50  * 1: 点 c 在有向向量 ab 的左侧 (逆时针)
51  * -1: 点 c 在有向向量 ab 的右侧 (顺时针)
52  * 0: 点 c 在直线 ab 上 (共线)
53  */
54 int to_left(const Point& a, const Point& b, const Point& c) {
55     ll res = cross(a, b, c);
56     if (res > 0) return 1;
57     if (res < 0) return -1;
58     return 0;
59 }
60
61 // 判断点 c 是否有向线段 ab 上 (闭区间)
62 bool is_on_segment(const Point& a, const Point& b, const Point& c) {
63     return to_left(a, b, c) == 0 && dot(c, a, b) <= 0;
64 }
65
66 // =====
67 // 3. 多边形模版 (Polygon)
68 // =====
69 struct Polygon {

```

```

70     vector<Point> p;
71
72     Polygon() = default;
73     Polygon(const vector<Point>& pts) : p(pts) {}
74
75     void add_point(ll x, ll y) {
76         p.emplace_back(x, y);
77     }
78
79     // 计算多边形有向面积的两倍
80     ll double_area() const {
81         ll res = 0;
82         int n = p.size();
83         for(int i = 0; i < n; i++) {
84             res += cross(p[i], p[(i + 1) % n]);
85         }
86         return abs(res);
87     }
88
89     // Andrew 算法求凸包
90     void get_convex_hull() {
91         if(p.size() <= 2) return;
92
93         sort(p.begin(), p.end());
94         vector<Point> hull;
95         int n = p.size();
96
97         for(int i = 0; i < n; i++) {
98             while(hull.size() > 1 && to_left(hull[hull.size() - 2],
hull.back(), p[i]) <= 0) {
99                 hull.pop_back();
100             }
101             hull.push_back(p[i]);
102         }
103
104         int t = hull.size();
105         for(int i = n - 2; i >= 0; i--) {
106             while(hull.size() > t && to_left(hull[hull.size() - 2],
hull.back(), p[i]) <= 0) {
107                 hull.pop_back();
108             }
109             hull.push_back(p[i]);
110         }
111
112         hull.pop_back();
113         p = hull;
114     }

```

```

115 };
116
117 void solve() {
118     int n, m;
119     cin >> n >> m;
120     Polygon S, T;
121
122     for(int i = 0; i < n; i++) {
123         ll x, y;
124         cin >> x >> y;
125         S.add_point(x, y);
126     }
127     for(int i = 0; i < m; i++) {
128         ll x, y;
129         cin >> x >> y;
130         T.add_point(x, y);
131     }
132
133     Polygon V;
134     for(int i = 0; i < n; i++) {
135         for(int j = 0; j < m; j++) {
136             Point shifted = T.p[j] - S.p[i];
137             V.add_point(shifted.x, shifted.y);
138         }
139     }
140     V.get_convex_hull();
141
142     db s1 = (db)S.double_area() / 2.0;
143     db s2 = (db)T.double_area() / 2.0;
144     db s3 = (db)V.double_area() / 2.0;
145     db res = (s1 * s2) / s3;
146     cout << fixed << setprecision(12) << res << '\n';
147 }

```

```

1  using ll = long long;
2  using db = long double;
3
4  // =====
5  // 1. 点与向量模版 (Point & Vector)
6  // =====
7  struct Point {
8      ll x, y;
9      Point(ll _x = 0, ll _y = 0) : x(_x), y(_y) {}
10
11      Point operator+(const Point& p) const { return Point(x + p.x, y + p.y); }
12      Point operator-(const Point& p) const { return Point(x - p.x, y - p.y); }
13

```

```

14     bool operator<(const Point& p) const {
15         if (x != p.x) return x < p.x;
16         return y < p.y;
17     }
18     bool operator==(const Point& p) const {
19         return x == p.x && y == p.y;
20     }
21 };
22
23 // =====
24 // 2. 核心几何函数
25 // =====
26 ll cross(const Point& a, const Point& b) {
27     return a.x * b.y - a.y * b.x;
28 }
29
30 ll cross(const Point& o, const Point& a, const Point& b) {
31     return cross(a - o, b - o);
32 }
33
34 ll dot(const Point& a, const Point& b) {
35     return a.x * b.x + a.y * b.y;
36 }
37
38 ll dot(const Point& o, const Point& a, const Point& b) {
39     return dot(a - o, b - o);
40 }
41
42 int to_left(const Point& a, const Point& b, const Point& c) {
43     ll res = cross(a, b, c);
44     if (res > 0) return 1;
45     if (res < 0) return -1;
46     return 0;
47 }
48
49 bool is_on_segment(const Point& a, const Point& b, const Point& c) {
50     return to_left(a, b, c) == 0 && dot(c, a, b) <= 0;
51 }
52
53 // =====
54 // 3. 多边形模版 (Polygon)
55 // =====
56 struct Polygon {
57     vector<Point> p;
58
59     Polygon() = default;
60     Polygon(const vector<Point>& pts) : p(pts) {}

```

```

61
62 void add_point(ll x, ll y) {
63     p.emplace_back(x, y);
64 }
65
66 // 重新排序：使最左下角的点成为起点，保证边向量从第一象限开始
67 void reorder() {
68     if(p.empty()) return;
69     int id = 0, n = p.size();
70     for(int i = 1; i < n; i++) {
71         if(p[i].y < p[id].y || (p[i].y == p[id].y && p[i].x < p[id].x)) {
72             id = i;
73         }
74     }
75     rotate(p.begin(), p.begin() + id, p.end());
76 }
77
78 ll double_area() const {
79     ll res = 0;
80     int n = p.size();
81     for(int i = 0; i < n; i++) {
82         res += cross(p[i], p[(i + 1) % n]);
83     }
84     return abs(res);
85 }
86
87 void get_convex_hull() {
88     if(p.size() <= 2) return;
89
90     sort(p.begin(), p.end());
91     vector<Point> hull;
92     int n = p.size();
93
94     for(int i = 0; i < n; i++) {
95         while(hull.size() > 1 && to_left(hull[hull.size() - 2],
hull.back(), p[i]) <= 0) {
96             hull.pop_back();
97         }
98         hull.push_back(p[i]);
99     }
100
101     int t = hull.size();
102     for(int i = n - 2; i >= 0; i--) {
103         while(hull.size() > t && to_left(hull[hull.size() - 2],
hull.back(), p[i]) <= 0) {
104             hull.pop_back();
105         }

```

```

106         hull.push_back(p[i]);
107     }
108
109     hull.pop_back();
110     p = hull;
111 }
112 };
113
114 // =====
115 // 4. 求闵可夫斯基和 (O(n+m))
116 // =====
117 Polygon minkowski(Polygon A, Polygon B) {
118     // 1. 找到各自的左下角基准点
119     A.reorder();
120     B.reorder();
121
122     int n = A.p.size(), m = B.p.size();
123     vector<Point> v1(n), v2(m);
124
125     // 2. 求出所有的边向量 (由于是逆时针凸多边形, 极角严格单调递增)
126     for (int i = 0; i < n; i++) v1[i] = A.p[(i + 1) % n] - A.p[i];
127     for (int i = 0; i < m; i++) v2[i] = B.p[(i + 1) % m] - B.p[i];
128
129     Polygon res;
130     // 起点是两个基准点的和向量
131     res.add_point(A.p[0].x + B.p[0].x, A.p[0].y + B.p[0].y);
132
133     // 3. 双指针归并 (按极角大小合并)
134     int i = 0, j = 0;
135     while (i < n && j < m) {
136         Point cur = res.p.back();
137         if (cross(v1[i], v2[j]) >= 0) { // v1[i] 在 v2[j] 的顺时针方向 (极角更小)
138             res.add_point(cur.x + v1[i].x, cur.y + v1[i].y);
139             i++;
140         } else {
141             res.add_point(cur.x + v2[j].x, cur.y + v2[j].y);
142             j++;
143         }
144     }
145
146     // 4. 将剩余的向量补齐
147     while (i < n) {
148         Point cur = res.p.back();
149         res.add_point(cur.x + v1[i].x, cur.y + v1[i].y);
150         i++;
151     }
152     while (j < m) {

```

```

153         Point cur = res.p.back();
154         res.add_point(cur.x + v2[j].x, cur.y + v2[j].y);
155         j++;
156     }
157
158     // 弹出最后一个点 (因为会与起点重合)
159     res.p.pop_back();
160     return res;
161 }
162
163 void solve() {
164     int n, m;
165     cin >> n >> m;
166     Polygon S, T;
167
168     for(int i = 0; i < n; i++) {
169         ll x, y;
170         cin >> x >> y;
171         S.add_point(x, y);
172     }
173     for(int i = 0; i < m; i++) {
174         ll x, y;
175         cin >> x >> y;
176         T.add_point(x, y);
177     }
178
179     // 为了求 T 和 -S 的闵可夫斯基和, 先构建 -S 多边形
180     Polygon neg_S;
181     for(auto pt : S.p) {
182         neg_S.add_point(-pt.x, -pt.y);
183     }
184     // 直接在 O(N+M) 时间内求解闵可夫斯基和
185     Polygon V = minkowski(T, neg_S);
186
187     db s1 = (db)S.double_area() / 2.0;
188     db s2 = (db)T.double_area() / 2.0;
189     db s3 = (db)V.double_area() / 2.0;
190     db res = (s1 * s2) / s3;
191     cout << fixed << setprecision(12) << res << '\n';
192 }

```